Agents

Language Models

Course Contents

Deep Learning
Transformer
LLMs
Context Engineering
Finetuning
Agents
The Bigger Picture

This Part: Agents

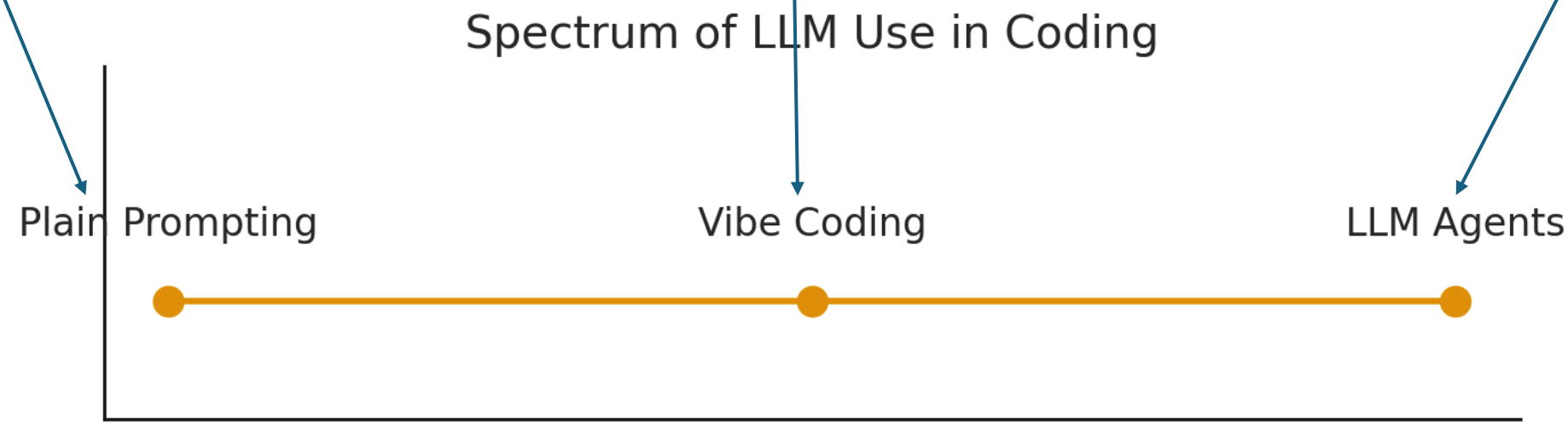
- tool use
- memory
- reasoning
- LLM orchestration
- planning
- multi-agent systems
- agentic decision layer
- local AI stack

Example: LLM Use in Coding

using ChatGPT or similar for snippets and Q&A

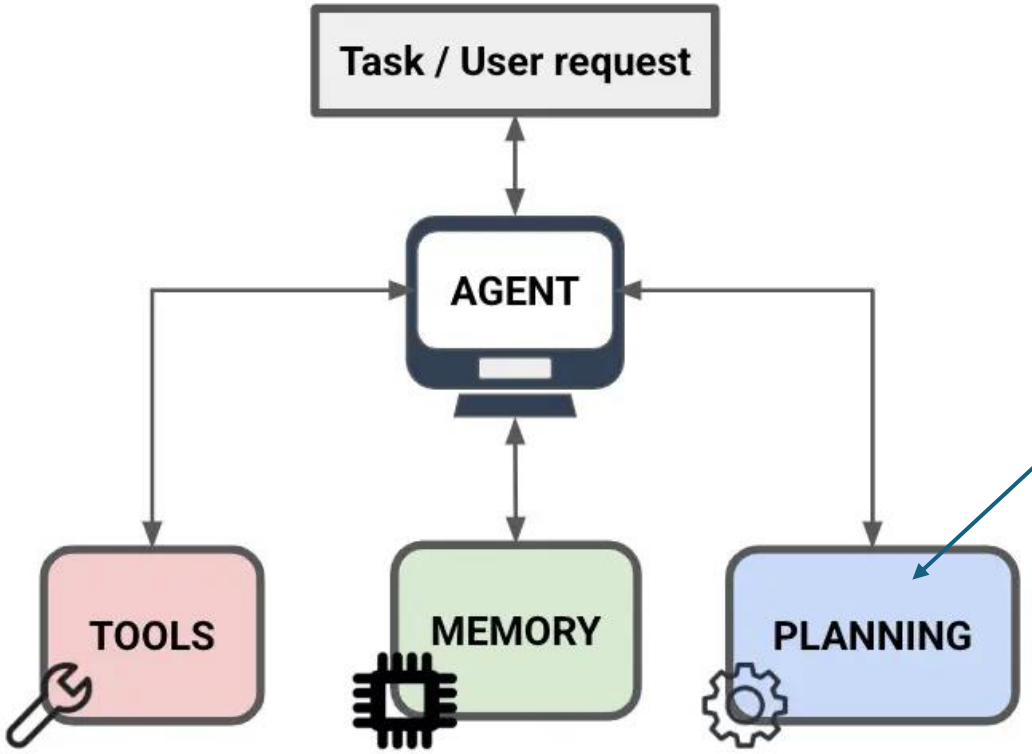
interactive pair-programming with tools like GitHub Copilot or Cursor

autonomous systems like AutoGPT plan and execute tasks with minimal human oversight



Agentic Intelligence

goal: autonomous problem-solving



decomposition of complex task in simpler steps (uses reasoning capabilities)

Intuitive and Analytical Thinking

two systems of human thinking (from Kahneman, *Thinking, Fast and Slow*):

- system 1: fast, automatic, intuitive, effortless, often subconscious
- system 2: slow, deliberate, analytical, effortful, conscious

LLMs analogous to system 1 (although without true intuition or understanding), lacking the depth of reflective, analytic reasoning

→ for agentic intelligence, need to enhance LLMs with system 2 capabilities, including tool usage and fact-checking

ReAct

prompting technique instructing an LLM to interleave reasoning steps (CoT-like) with actions (tool use):

- reasoning (Re): model reflects, analyzes the problem, breaks it down, and plans
- acting (Act): model performs an external action (e.g., querying a database, calling an API, using a calculator, searching the web)
- process loops: reasoning → action → observation → updated reasoning → next action → … until the task is completed

ReAct Prompt Template

zero-shot ReAct:

```
You are an intelligent assistant that can reason and take actions.
For each step, follow this format:

Reasoning: [your thought process here]
Action: [the action you will take, if any]
Observation: [the result of the action]

Repeat as needed until you can confidently provide the answer.
At the end, give your final answer in this format:

Final Answer: [your final response here]

---

Task: [insert the problem/question here]
```

often few-shot ReAct: include 1–3 worked-out examples of ReAct loop

→ improves reliability on more complex, multi-step tasks.

ReAct Example in Use

⋮
Task: What is the square root of the population of Tokyo (approximately)?

response of the model:

Reasoning: I need to find the population of Tokyo first, then compute its square root.
Action: Search "Tokyo population 2023"
Observation: Tokyo's population is about 37.4 million.

Reasoning: Now I calculate the square root of 37.4 million.
Action: Calculator: sqrt(37,400,000)
Observation: Result is about 6116.

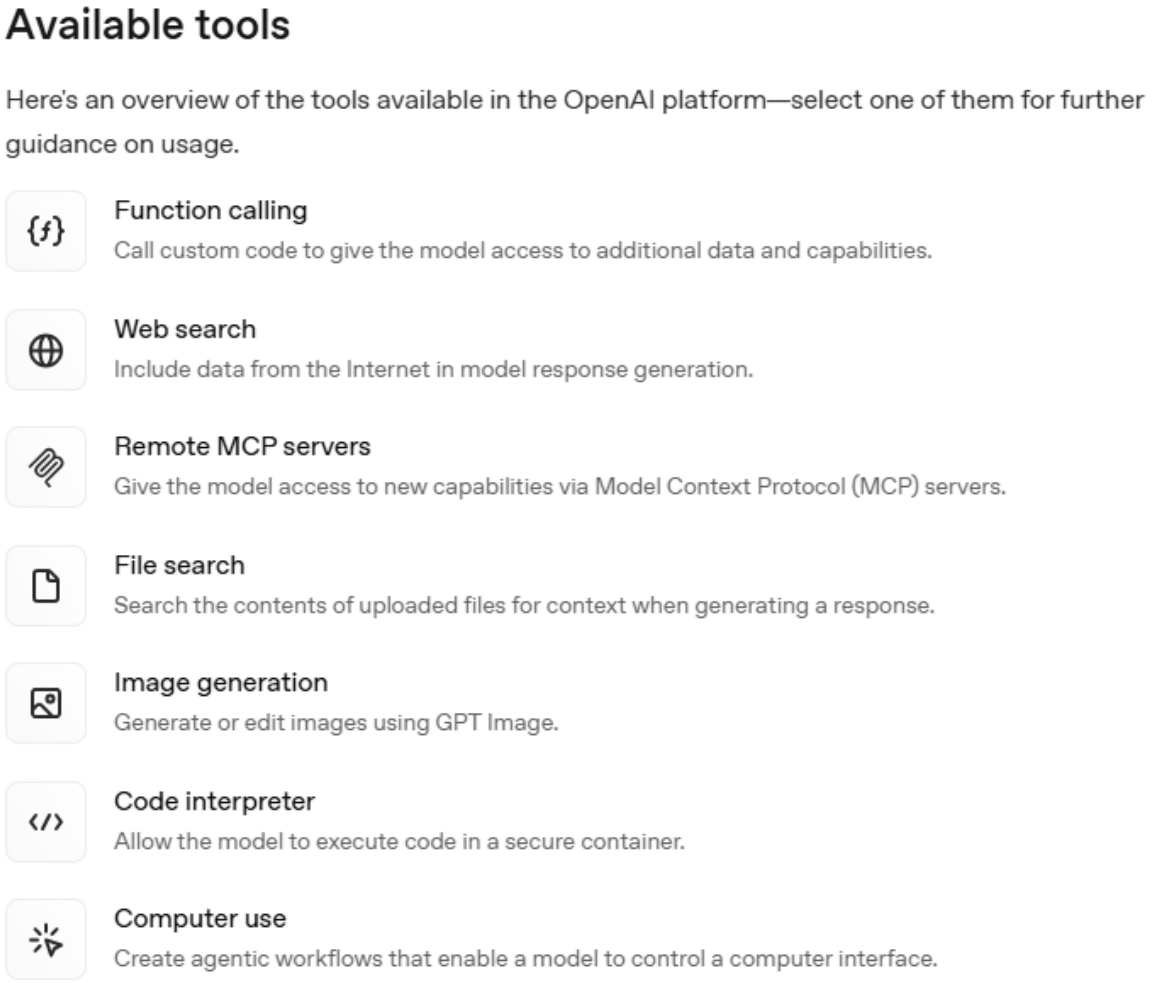
Final Answer: The square root of Tokyo's population (~37.4 million) is approximately 6116.

Tool Usage

crucial LLM capabilities:

1. know how to access different tools
   - code interpreters
   - web search
   - calculators (to overcome lack of mathematical understanding)
   - …
2. select right tool at right time

example: OpenAI



Acquistion of Tool Usage Skills

using instruction-tuned LLMs (like ChatGPT) as agent:

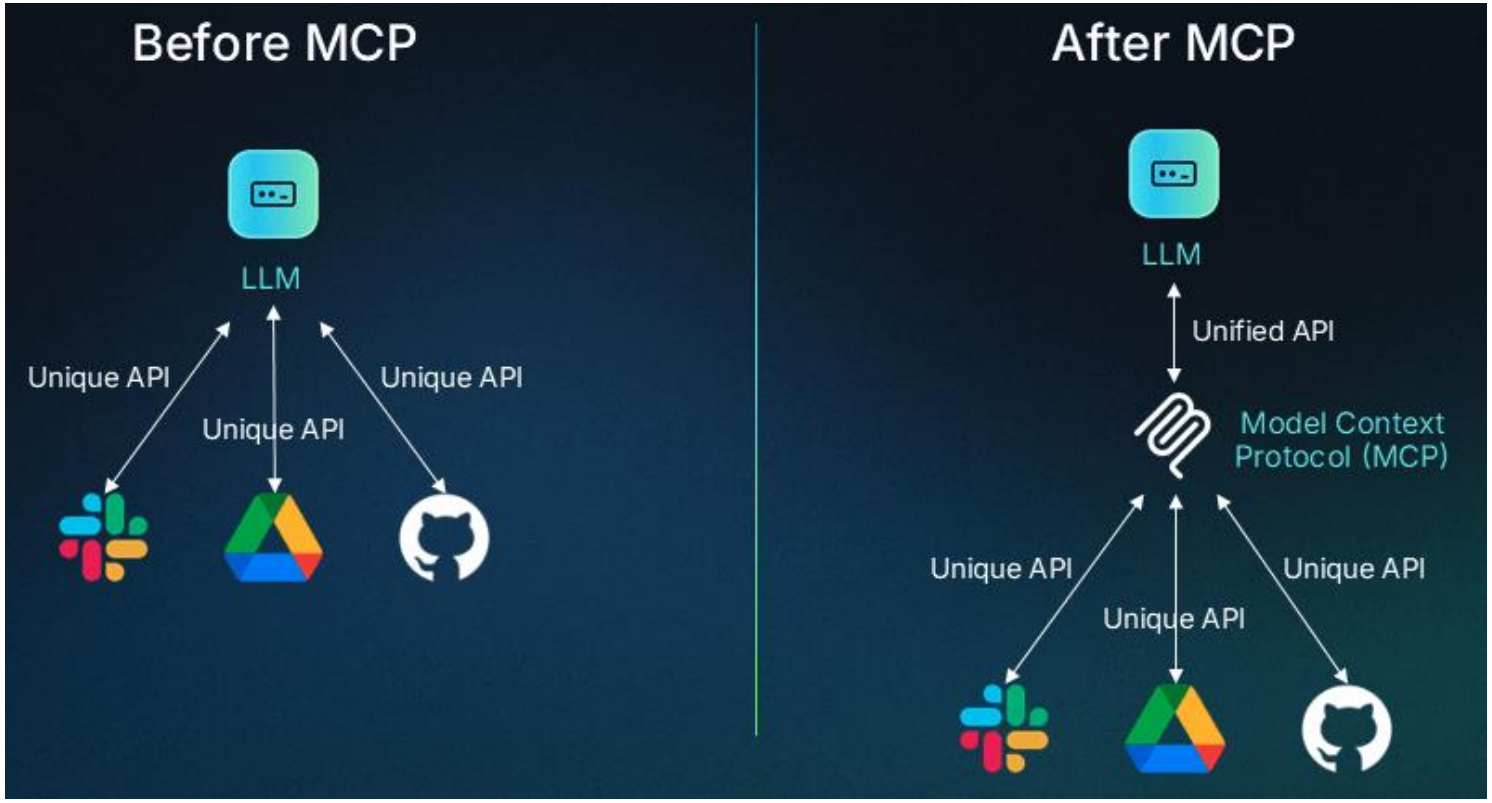
- core capability (*how* to use a tool) comes from pretraining and different finetuning stages
- tool access controlled at runtime via prompting and system messages
- tool selection (*when* to use *which* tool) decided from context

potentially, custom finetuning for specific tools or workflows

Model Context Protocol (MCP)

open protocol to connect LLMs to different tools or data sources

standardization for provisioning of external context



Tool Injection

LLMs (like Claude or Qwen) have no inherent knowledge of your local setup, installed extensions, or MCP servers

→ need for a prompt-engineering mechanism that dynamically teaches the LLM what it can do before the conversation starts

1. discovery: client (e.g., Cline) queries all active MCP servers for their available tools and capabilities
2. injection: client formats these tools (names, descriptions, arguments) and injects them directly into the system prompt.
3. decision: LLM reads the user prompt, realizes it needs external data, and responds with a structured code block (e.g., a JSON object requesting fetch_webpage)
4. interception: client intercepts the LLM's JSON response instead of showing it to the user
5. execution: client runs the tool via the MCP server and feeds the raw result back into the LLM's chat history to continue the loop

Tool Confusion & Context Bloat

more tools ≠ a smarter agent: blindly adding dozens of MCP servers to your setup will actively degrade the agent's performance

root cause: every added tool increases the cognitive load on the LLM and consumes limited memory resources

- tool confusion: as the number of available tools grows, the LLM struggles to pick the exact right one
- context bloat: tool definitions occupy valuable space inside the context window

- frontier models can reliably handle 20 to 30 tools simultaneously
- smaller models often start losing accuracy or failing formatting constraints at just 8 to 10 tools

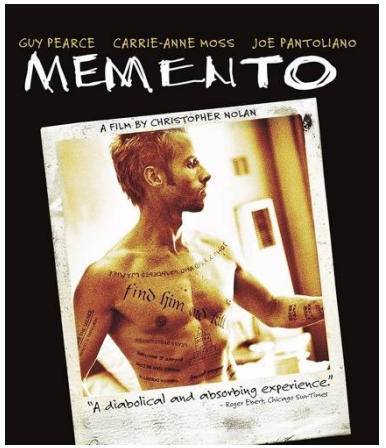
Test-Time Scaling

improve a model's performance (especially reasoning capabilities) without retraining it, by allocating more compute during inference time:

- more complex prompting strategies (CoT, self-consistency, prompt optimization, …)
- extensive tool use

→ test-time compute (number of samples, reasoning steps) as another scaling law (in addition to parameters, training data, training compute)

LLM/Agent Memory



short-term memory: context

- chatbot conversation: typically just append each turn
- for conversations longer than context window, older turns need to be summarized or truncated
- forgotten after each conversation

long-term memory: survives across conversations

- typically implemented through external vector store and context injection → RAG mechanism
- expensive alternative: per-instance/per-agent adaption (test-time training) → only update (i.e., finetune) small adapters (e.g., LoRA)

Reasoning Models

some reasoning capabilities:

standard LLMs using prompting strategies such as CoT or ReAct

→ reasoning emerges implicitly from pretraining (performance can be brittle and highly prompt-dependent)

better reasoning capabilities:

reasoning-specific LLMs explicitly trained or finetuned on reasoning objectives (often with CoT-style supervision or multi-step reasoning datasets)

→ learn to reason by default, not just when prompted

examples: OpenAI o1, DeepSeek-R1 (open source)

LLM Orchestration and Agents

wrappers around LLM calls to make them more useful in real-world applications

LLM orchestration:
pipelines + prompt templates + access to tools & memory + context management
→ structured environment
e.g., LangChain, LlamaIndex

agent systems:
LLM orchestration + autonomy/planning loop
→ kind of autopilot on top of orchestration layer
e.g., AutoGPT, OpenAgents, BabyAGI

Planning

scaffolding around (reasoning-strong) LLM, structuring acting in sequence:

- break goals into subgoals
- use the LLM to propose or refine steps
- optionally search through multiple alternatives
- execute actions and adapt plans dynamically

implementation options:

- use LLM itself as a planner via prompting (CoT, ReAct)
- search-based planning
- control loop (LLM as policy generator) → AutoGPT, etc

Multi-Agent Systems

idea:

complex, distributed, and dynamic problems can be solved more efficiently, flexibly, and robustly by using multiple specialized agents rather than a single one

→ extended agent system:

LLM orchestration + autonomy/planning loop

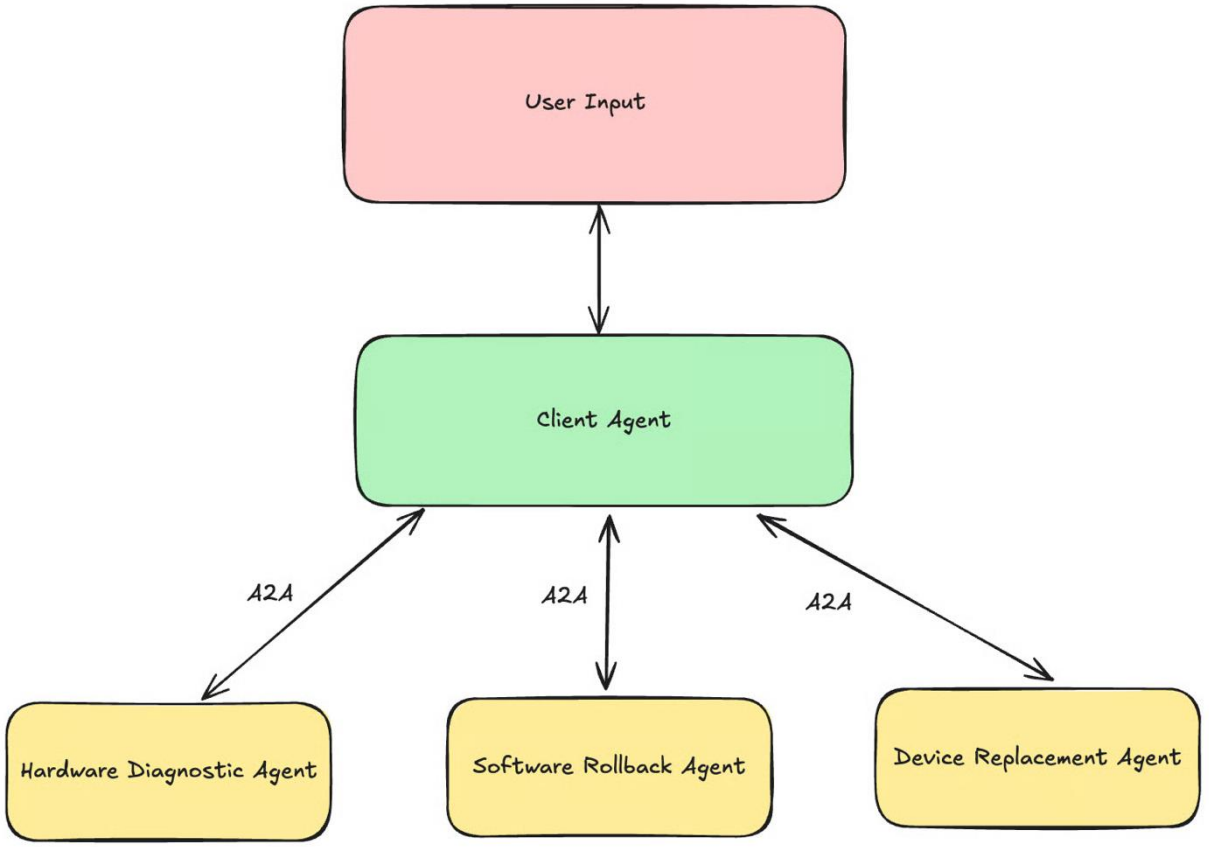
+ coordination between multiple agents

e.g., AutoGen

Agent2Agent Protocol (A2A)

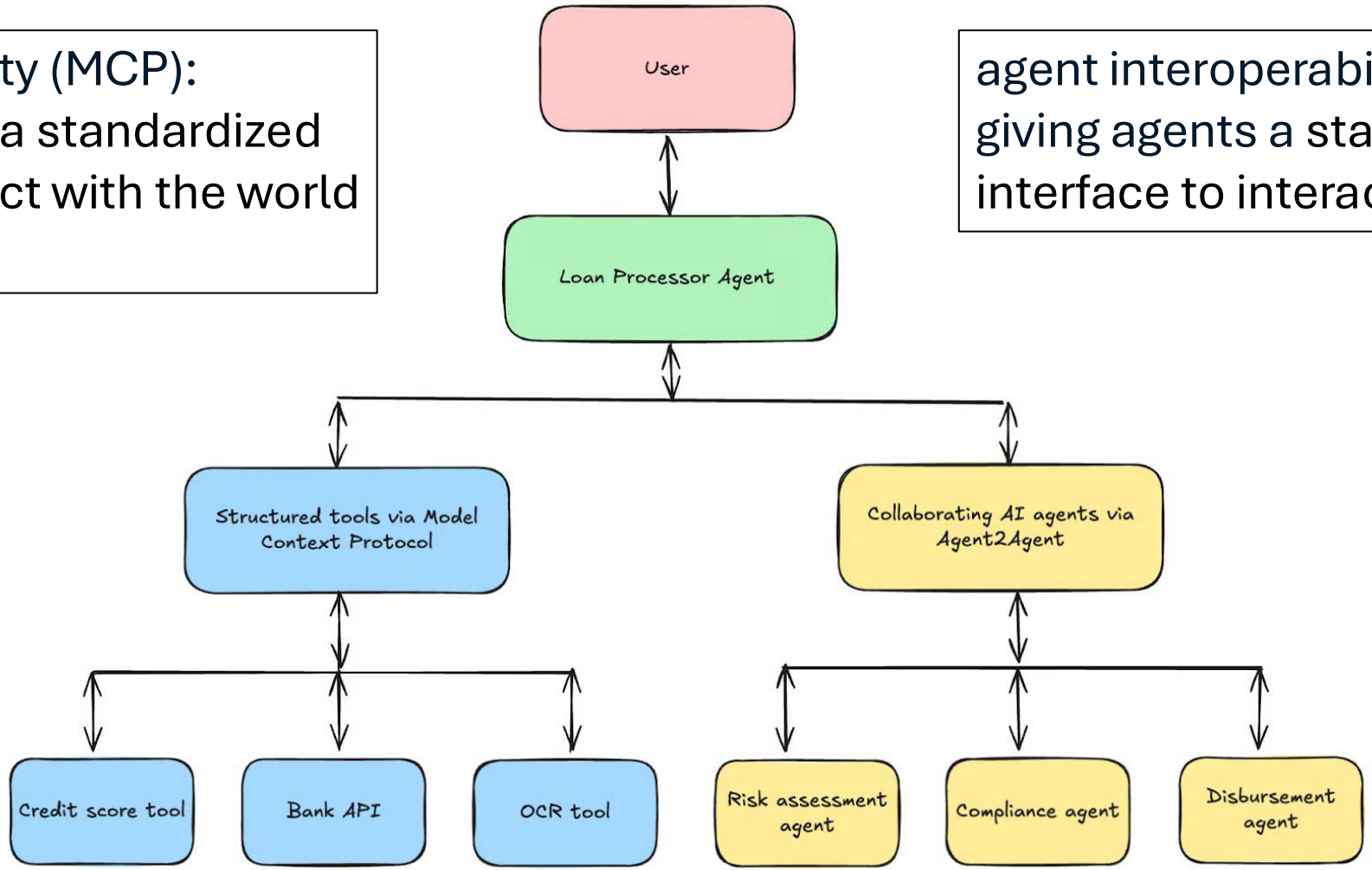
open standard for enabling communication, collaboration, and interoperability between autonomous AI agents (complementary to MCP)

example:



Example for Combination of A2A and MCP

tool interoperability (MCP):
giving each agent a standardized interface to interact with the world (tools, data, APIs)



agent interoperability (A2A):
giving agents a standardized interface to interact with each other

Agentic Decision Layer: System-One Models

problems with frontier LLMs in agent loops:

- overkill: using a 100B+ parameter generative model to make simple if/else decisions
- bottlenecks: sequential token-by-token generation is slow (seconds) and costly
- risks: hallucinations or syntax errors break structured tool calls (json parsing errors)

Jev (by TypeSafe AI):

- a non-generative, "system one" discriminative model focused purely on classification
- no text generation: input a state (up to 64k tokens) and questions, output exact typed answers with calibrated probability metrics (Noul, Choice, Score)
- up to 200x faster inference than traditional LLMs, very cost efficient (highly parallelized)

Jev Architecture: Encoder-Based Zero-Shot Classifier

eliminating autoregression

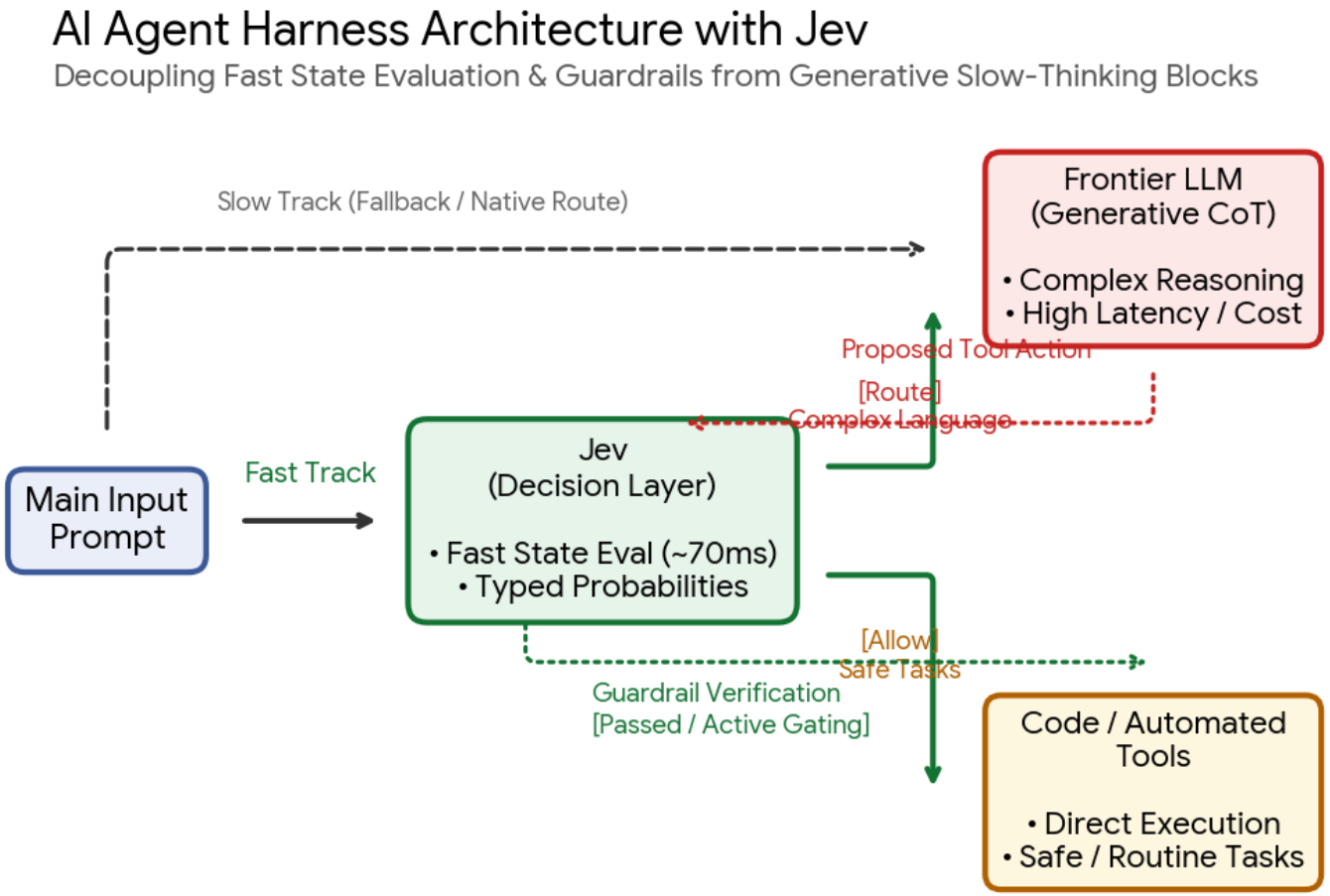
- **single forward pass:** processes text, questions, and options simultaneously using a bidirectional transformer encoder and a parallel sampler
- **micro-output heads:** uses hyper-focused mathematical readout heads (*Choice, Score, Noul*) limited to 255 discrete options per question

dynamic input classes

- **cross-attention alignment:** instead of hardcoded classification layers, the model maps the text's vector space directly against the options provided in the prompt
- **instant adaptation:** understands complex, natural-language logical rules and updates routing instantly at inference time without any finetuning
- **decision-centric pretraining:** trained on trillions of synthetic choice matrices to master the meta-skill of categorical mapping rather than text composition

statistically calibrated outputs

- **RLCD training:** leverages *Reinforcement Learning for Calibrated Decisions* to penalize overconfidence and ensure statistical accuracy
- **trustworthy thresholds:** a 70% confidence output translates directly to a ~70% real-world success rate, enabling reliable fallback thresholds in production code



agentic routing:

instantly decides which specialist agent or API tool should handle an incoming prompt

tool-Call guardrails (gating):

evaluates a proposed tool payload (e.g., a bash command) before execution → returns a safe allow/block/review signal

RAG validation:

instantly judges if retrieved chunks are relevant before feeding them into the generator

Golden Rule of Agent Efficiency

use Frontier LLMs as the *Brain*

(only when you genuinely need deep reasoning, creativity, or human-like synthesis)

use system-one models like Jev as the *Nervous System*

(for the hundreds of micro-decisions, routing logic, and safety guardrails that happen under the hood)

Don't waste cognitive power and compute money on tasks that can be solved with calibrated probabilities.

Build hybrid, fast, and deterministic architectures.

Local AI Agent Stack

infrastructure & base models:

- Ollama: hosts open local LLMs (e.g., Llama 3, Qwen 2.5, Gemma) via CLI/API (alternative: LM Studio)

user interfaces & knowledge:

- Open WebUI: feature-rich, ChatGPT-like frontend for local models (alternative: Jan)
- AnythingLLM: ready-to-use private RAG & document knowledge base (alternative: PrivateGPT)

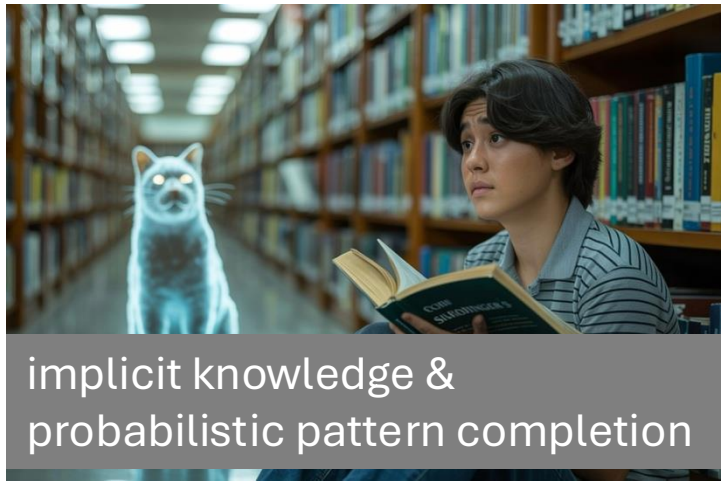
developer copilots & coding agents:

- Continue: inline code autocomplete & side-panel chat inside VS Code (with Ollama)
- Cline: full autonomous coding agent inside VS Code (creates files, runs terminal)
  (alternatives: Aider (terminal-based), Goose (desktop app / MCP-ready))
- Codex (free edition): powerful free coding engine combined with Ollama's local LLMs

advanced multi-agent & OS frameworks:

- AutoGen: native multi-agent planning, workflows & collaboration (alternative: AutoGPT)
- Open Interpreter: full natural language control over your local terminal & environment
- OpenClaw: advanced cross-platform computer use agent via screenshot-action-loops

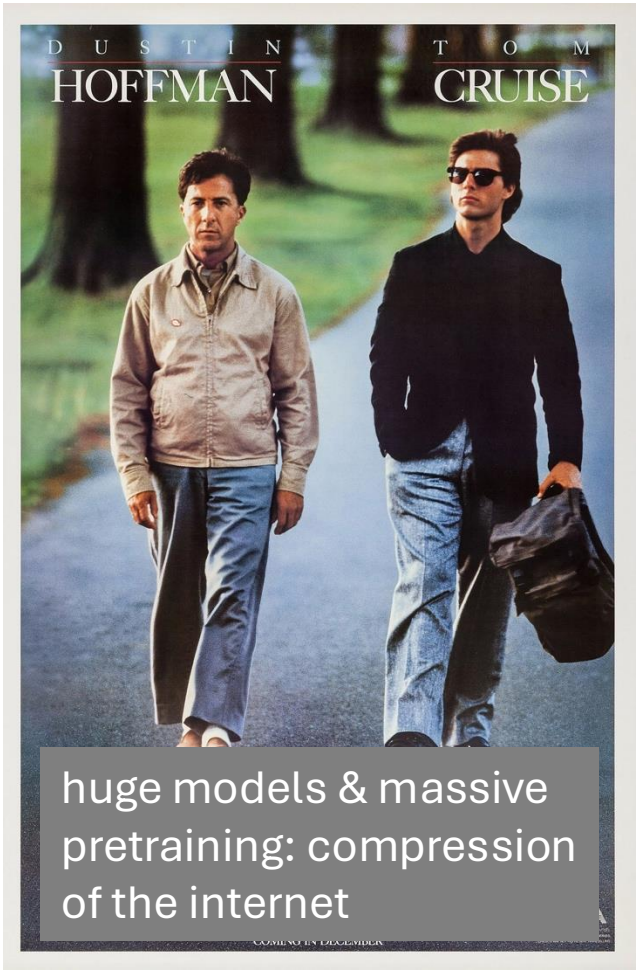
hallucinations



implicit knowledge & probabilistic pattern completion

LLM: *kind of a lossy simulation of a savant with cognitive issues*

encyclopedic knowledge

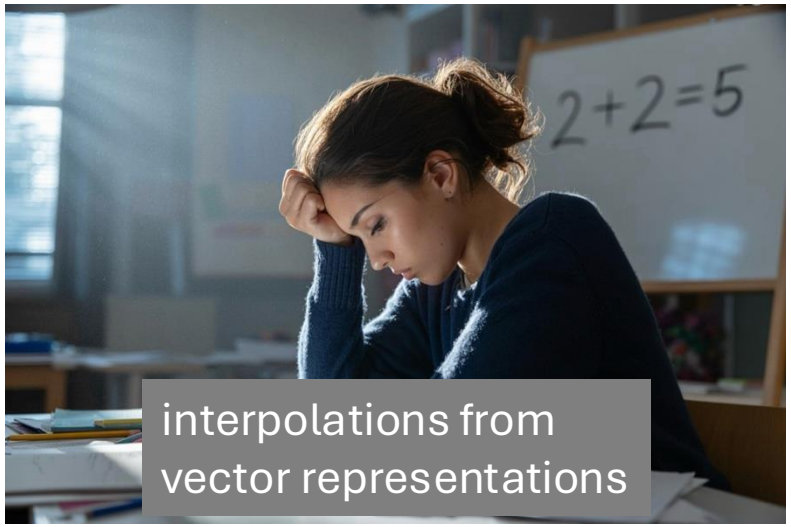


huge models & massive pretraining: compression of the internet



only short-term memory in context

anterograde amnesia



interpolations from vector representations

jagged intelligence

from Karpathy

Exercise: Tool Use & Planning

Build a simple LLM agent:

- choose an agentic task (math, data analysis, file organization, trip planning, …)
- define the agent & planning strategy (prompt templates, tool call and planning instructions)
- provide tools the agent can call (functions, APIs, calculator, …)
- implement the reasoning/planning loop (ReAct, reflexion, …)
- optionally add short-term memory (scratchpad or state tracking)
- evaluate robustness and correctness (planning and reasoning quality, task success rate, …)